Block guide

Eleven blocks, in build order. What to do, and how you know it worked.

```
220  records in schematic.json
 -42  mechanical filler   (blanking covers, wire ridges, spacers, insulators)
  -5  "- Kombination"     (assembly wrappers duplicating a real child)
_____
173  core
      ├─  92  devices with a unique tag   →  92 checklist items
      └─  81  terminals across 8 strips   →   8 checklist items
_____
100  checklist items
```

Two rules

Do not start a block until the one before it holds. Every block has a gate — if the gate has not passed, the block is not finished, however much code exists.

Start at Block 5. It costs one afternoon and it can invalidate the method. Everything else is built on the assumption it answers well.

SET THE FACTS	Block 0 — the spec everything is sized off
KNOW YOUR DATA	Blocks 1–3 — no microphone needed
TEST WHAT COULD KILL IT	Blocks 4–5 — the risk, taken early
BUILD THE JUDGEMENT	Blocks 6–8 — the system itself
PROVE IT	Blocks 9–10 — the thesis chapter

Blocks 0, 3, 5 and 9 produce no code at all. They are a document, a meeting, a measurement and an experiment. Log every block in `DECISIONS.md`: what you expected, what happened, what you changed.

SET THE FACTS

0 Fix CONTEXT.md

DO THIS

1. Replace `CONTEXT.md` with v2.0 inside `Redlining/`, not beside it.
2. Read §12, the change log, end to end. It is the summary of everything that moved.
3. Read §8 twice. It is the section your advisor will push back on.
4. Send v2.0 to your advisor before Block 3 — §7 sets up the banding session.
5. Create `DECISIONS.md` and write the Block 0 entry.

SUCCESS LOOKS LIKE

- [ ] No sentence in the file contradicts the export or the photographs.
- [ ] Someone who has never seen the project can read it and brief you correctly.
- [ ] You can defend every `DECISION` by its stated reason, not by "we chose that".
- [ ] The file names what the thesis *cannot* claim, not only what it can.

WATCH FOR

- "~50–60 components" left anywhere. It invalidates the time budget, the advisor session and the defer ceiling.
- Treating this as admin. It is the spec — everything downstream is sized off it.

Gate · **the file no longer contradicts the design.**

1

KNOW YOUR DATA

Loader

DO THIS

1. Put `schematic.json` in `data/`.
2. Read it and iterate the 220 component records.
3. Drop the 42 mechanical filler parts: blanking covers, wire ridges, `Distanzstück`, `Isolierstück`.
4. Drop the 5 `- Kombination` wrapper records.
5. Key each record on `(designation, location)` — never on designation alone.
6. Print the counts and compare them against §6 of CONTEXT.md.
7. Log the entry in `DECISIONS.md`.

SUCCESS LOOKS LIKE

- [ ] Exactly **173** core records.
- [ ] Exactly **37** distinct part numbers.
- [ ] **92** uniquely-tagged devices; **81** terminals across **8** strips.
- [ ] Every `- Kombination` record gone, and its real child still present.
- [ ] Running it twice gives identical numbers.

WATCH FOR

- `-1Q2` and `-1Q3` each appear twice — once as a wrapper, once as the real device. Keep the record that carries a part number.
- Two records have an empty `location`. Decide what happens to them and write the decision down.
- Never drop anything silently. Count what you drop, and why.

Gate · **173 core parts and 37 part numbers, reproducibly.**

2

KNOW YOUR DATA

Position — coordinates into spoken places

DO THIS

1. Group records by frame: `+AA00001VLXX` and `+AA00005VLXX`.
2. Band by `relative_position.y` — a band is a rail.
3. Sort by `relative_position.x` within the band — that is the position along the rail.
4. Name the rails from the `layer` field (`AB`, `JB`, `QA`, `CA`, `DA`).
5. Print the full walking order on paper.
6. Stand at the cabinet with the printout and check it item by item.

SUCCESS LOOKS LIKE

- [ ] The order matches what you see — left to right, top to bottom.
- [ ] Terminal pitch comes out as a constant 5.2 mm.
- [ ] Every one of the 100 items has a location a person can say out loud.
- [ ] You walk the cabinet once, with no backtracking.

WATCH FOR

- Do not trust the derived order until you have checked it by eye. This is the only module the schematic cannot verify for you.
- The two frames may be two doors or two plates. Confirm which one the trainee faces first.
- If a rail name is hard to say or hear, rename it. The trainee hears it once and must find the spot.

Gate · **the walking order matches the real cabinet.**

KNOW YOUR DATA

3 Checklist and priority bands

DO THIS

1. Build the 100 items: 92 devices plus 8 strips.
2. List the 21 device types and 8 strips as 29 rows in `data/bands.csv`.
3. Book the advisor session. Bring the 29 rows, not the 100 items.
4. Band each row: 1 protective · 2 main power path · 3 control and signal · 4 cosmetic.
5. Inherit each band to every component of that type.
6. Sort the checklist: band first, physical position within band.
7. Record who decided, and when.

SUCCESS LOOKS LIKE

- [ ] `bands.csv` has 29 rows and every one is banded.
- [ ] All 100 checklist items carry a band.
- [ ] Band 1 comes first and genuinely holds the protective devices — RCDs, the surge arrester, the main switch.
- [ ] Your advisor's name and the date are in `DECISIONS.md`.

WATCH FOR

- Do not band 100 components by hand. Twenty-nine decisions, inherited — that was the point of ranking by type.
- If one individual device truly needs a different band from its type, that is a real exception. Write down why.
- Decide whether the 10 `ZEW 35 DBS` end brackets count toward strip totals. CONTEXT §10 still lists this as open.

Gate · **all 100 items carry a band.**

TEST WHAT COULD KILL IT

4 Normaliser

DO THIS

1. Record yourself reading 20 real labels at the cabinet.
2. Transcribe them and keep the raw text exactly as it came out.
3. Write plain rules: strip spaces, letters to upper case, spelled digits to numerals.
4. Test: `"a nine f zero three one one six"` → `A9F03116`.
5. Test the rating line: `"i c sixty n b sixteen"` → `iC60N B16`.
6. Keep it as rules. No model belongs in this module.

SUCCESS LOOKS LIKE

- [ ] All 20 real reads normalise to the correct string.
- [ ] It is deterministic — the same input always gives the same output.
- [ ] A failure produces a readable reason, not a crash.
- [ ] It runs with no network and no API key.

WATCH FOR

- Build the rules from your own recordings, not from what you imagine speech looks like.
- Never "fix" a read by snapping it to the nearest legal part number. That is the adjudicator's job, and doing it here launders the defect you are hunting.
- Always keep the raw transcript alongside the normalised one.

Gate · **it survives 20 real recorded reads.**

START HERE

5

TEST WHAT COULD KILL IT

ASR reality check

DO THIS

1. Go to the cabinet with a phone.
2. Read 30 real labels aloud — part number and rating line for each.
3. Include the `B10` devices sitting among the `B16` devices (`-8F7`, `-8F8`).
4. Transcribe with the Whisper API.
5. Transcribe the same audio with local `faster-whisper`.
6. Count how many normalise to the correct string, per engine.
7. Note the ambient noise conditions.
8. Write the error rate into `DECISIONS.md`.

SUCCESS LOOKS LIKE

- [ ] You have a number on paper, for each engine.
- [ ] You know whether `A9F03116` survives as itself, and whether B16 and B10 stay apart. That one character is the whole point.
- [ ] You know whether local transcription is good enough — which keeps recording in the lab and closes the works-council question.
- [ ] If the error rate is poor, you stop and talk to your advisor before building anything else.

WATCH FOR

- This block can invalidate the method. That is exactly why it comes first.
- A bad result is a result. It is a thesis finding, not a failure.
- Do not read the labels from memory. Read what is printed, even when you are sure you know it.

Gate · **you have an error rate on paper.**

6

BUILD THE JUDGEMENT

Adjudicator

DO THIS

1. Take normalised strings as input. No audio enters this module.
2. Compare the part number against the expected value.
3. Compare the rating line against the expected value.
4. Cross-check the two readings against each other.
5. Return one of four outcomes: match · mismatch · not-in-schematic · abstain.
6. Attach a plain-language reason to every outcome.
7. Write tests with typed input, including `A9F03310` where `A9F03116` is expected.

SUCCESS LOOKS LIKE

- [ ] All four outcomes are reachable by a test.
- [ ] A part number matching no schematic entry returns *not-in-schematic* — never a nearest match.
- [ ] Two clean reads that disagree return *mismatch*, not a third attempt.
- [ ] The reason string is something a reviewer can read and act on.
- [ ] It runs offline, in milliseconds, with no microphone.

WATCH FOR

- Only 37 legal part numbers exist in this cabinet. Do not fuzzy-match into that set — that is the laundering CONTEXT §7 forbids.
- Separate "misheard a legal part" from "a genuinely different part". At small edit distances these look identical, and that case is *abstain*.
- Strips are compared as counts, not strings. That is a different path through the module.

Gate · **every outcome reachable by a test.**

7

BUILD THE JUDGEMENT

Session runner

DO THIS

1. Walk the checklist in band-then-position order.
2. Prompt by location only. Never name the expected device.
3. Take the read and commit it before judging.
4. On abstain, re-ask silently: "please read it again", with no echo of what was heard.
5. Allow at most two re-asks, then flag the item and move on.
6. Reveal the expected value only after the verdict is fixed.
7. Log every attempt with its attempt number.
8. Complete one full 100-item run.

SUCCESS LOOKS LIKE

- [ ] One uninterrupted run, start to finish.
- [ ] The system never reveals what it expects before the trainee commits.
- [ ] Re-asks never echo the heard value.
- [ ] No item consumes more than three attempts.
- [ ] Flags queue up and the run keeps moving.
- [ ] You know how long the run took.

WATCH FOR

- Any hint before commit destroys the confirmation-bias protection. This is the point of the entire design.
- Three attempts on one item is itself a signal — an illegible label, or noise. Log it as one.
- Time the run. If it runs past an hour, the 10% defer ceiling becomes a real problem.

Gate · **one complete 100-item run.**

BUILD THE JUDGEMENT

8 Flags and report

DO THIS

1. Save every attempt: item, raw transcript, normalised value, confidence, verdict, audio path.
2. Retain the audio for each flag.
3. At the end of the run, walk the flag queue.
4. Let the trainee annotate only — a note, and "I misspoke" versus "the part looks wrong".
5. Provide no way to close or delete a flag.
6. Emit all 100 items, not only the flagged ones.
7. Write to `runs/`, gitignored, and copy one sample run into `tests/` as a fixture.

SUCCESS LOOKS LIKE

- [ ] The file contains all 100 verdicts.
- [ ] Each flag carries both the original read and the trainee's account of it.
- [ ] Nothing can be removed — check the code has no delete path at all.
- [ ] A reviewer could act on the file without asking you a single question.
- [ ] The audio behind any flag can be replayed.

WATCH FOR

- CONTEXT §4: a trainee closing a flag is a trainee performing sign-off. The system must make it impossible, not merely discouraged.
- The matches must be in the file. Without them there is no false-flag rate.
- Never commit `runs/`. It holds voice recordings.

Gate · **a reviewer could act on the file unaided.**

PROVE IT

9 Evaluation

DO THIS

1. Write a candidate fault list: B16 swapped for B10, a wrong part, a missing device, a wrong terminal colour.
2. Select the actual faults with a random seed, and do not look at which were chosen.
3. Plant them in the cabinet.
4. Run the system end to end.
5. Count: caught · missed · falsely flagged · abstained.
6. Record the time per cabinet.
7. Write the limitation paragraph: you planted and ran alone, with no blinding.

SUCCESS LOOKS LIKE

- [ ] A per-item detection rate on the planted fault set.
- [ ] A false-flag rate.
- [ ] An abstain rate, compared against the 10% ceiling.
- [ ] A time per cabinet.
- [ ] The unblinding limitation is written down, not merely implied.
- [ ] You can state clearly which classes of defect this method cannot see at all.

WATCH FOR

- You are measuring yourself. Every accuracy number is bounded by that — say so in the chapter.
- CONTEXT §8 dropped the manual baseline, so you cannot claim "better than manual". Do not imply it either.
- A missed fault is data. Do not re-run until the numbers look better.

Gate · **the numbers exist and the limitation is written.**

CUT THIS FIRST

10

PROVE IT

Redlines

DO THIS

1. Take the confirmed mismatches from the run report.
2. Emit a structured redline for each: item key, what the schematic says, what the cabinet says, and the evidence.
3. Stop there. A human carries the redline upstream; there is no write-back to ECAD.

SUCCESS LOOKS LIKE

- [ ] A schematic owner could correct the drawing from the file alone.
- [ ] No automated write-back exists — CONTEXT §10 names the carried report as the realistic deliverable.

WATCH FOR

- This is the first thing to cut if the month runs short. Nothing else depends on it, and the thesis stands without it.

Gate · **a redline a human can act on — or a deliberate decision to skip it.**